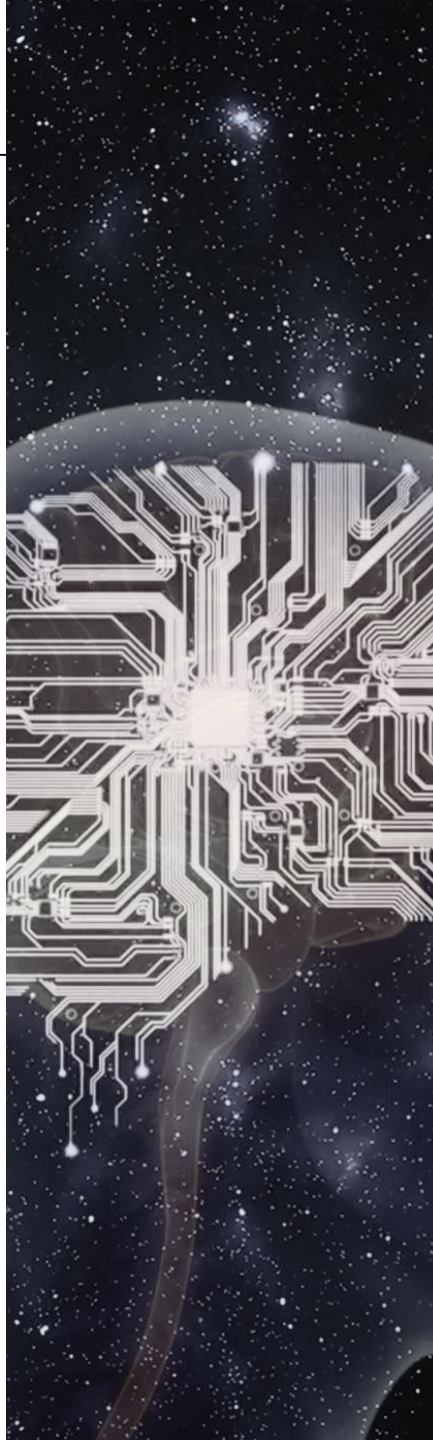
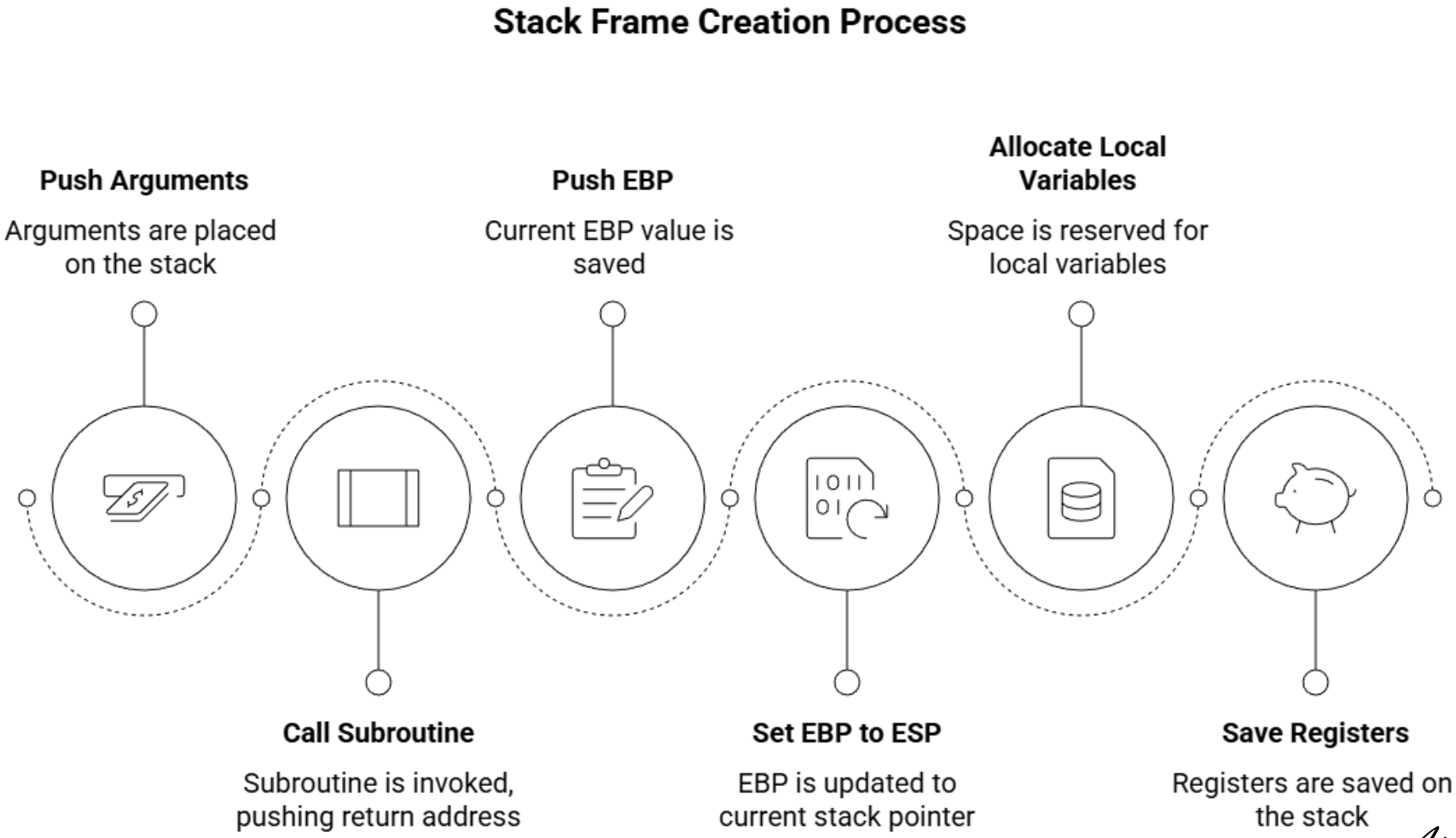
Assembly Programming

8. Advanced Procedures



Stack Parameters

A stack frame (or activation record) is the area of the stack set aside for passed arguments, subroutine return address, local variables, and saved registers.



Disadvantages of Register Parameters

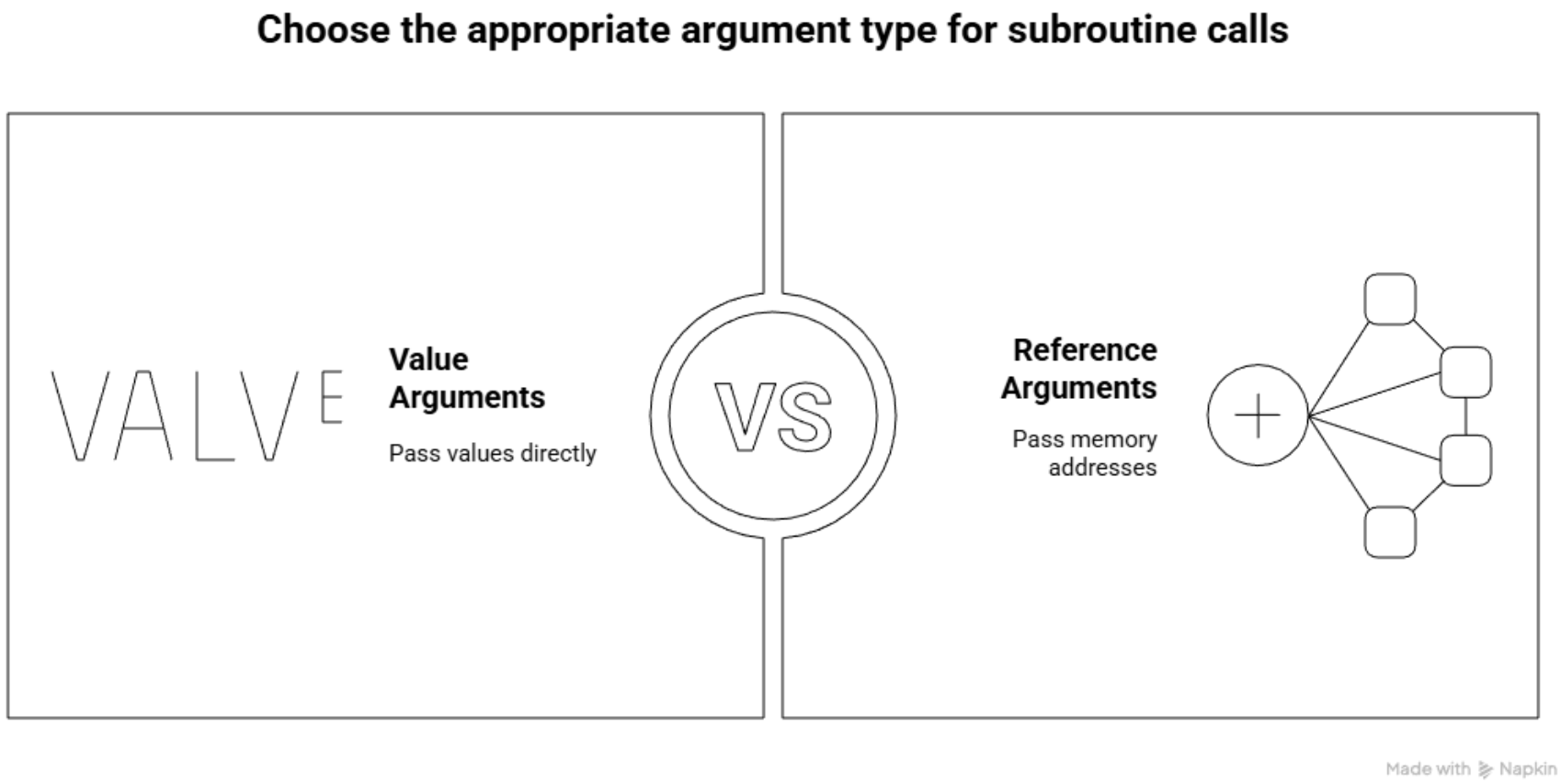
Any registers used as parameters must first be pushed on the stack before procedure calls, assigned the values of procedure arguments, and later restored to their original values after the procedure returns.

```
push  ebx                          ; save register values
push  ecx
push  esi
mov   esi,OFFSET array             ; starting OFFSET
mov   ecx,LENGTHOF array           ; size, in units
mov   ebx,TYPE array               ; doubleword format
call  DumpMem                      ; display memory
pop   esi                          ; restore register values
pop   ecx
pop   ebx
```

Not only do all the extra pushes and pops create code clutter, they tend to eliminate the very performance advantage we hoped to gain by avoiding register parameters!

Microsoft has included a parameter passing convention in 32-bit programs named fastcall.

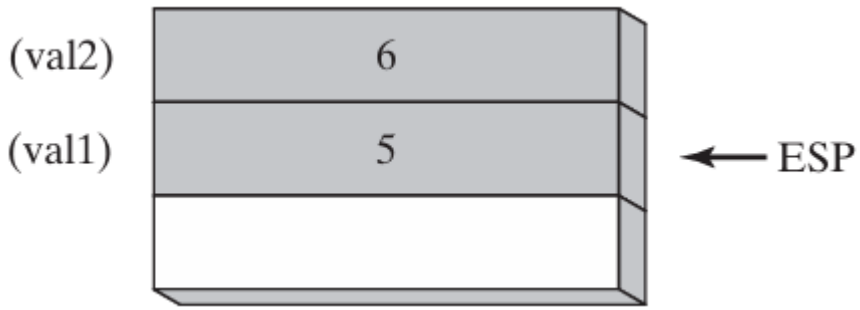
Stack Frames



Passing by Value

When an argument is passed by value, a copy of the value is pushed on the stack.

```
.data
val1   DWORD 5
val2   DWORD 6
.code
push   val2
push   val1
call   AddTwo
```

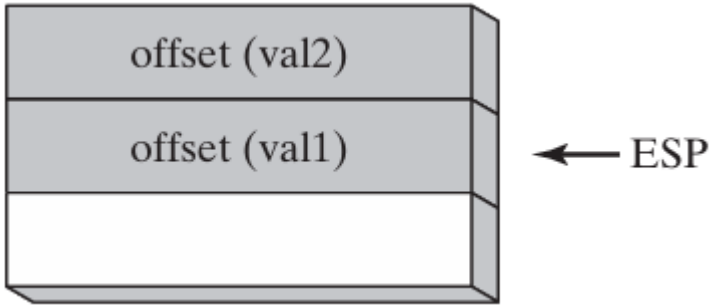


int sum = AddTwo(val1, val2);

Passing by Reference

An argument passed by reference consists of the address (offset) of an object.

```
push   OFFSET val2
push   OFFSET val1
call   Swap
```



Swap(&val1, &val2);

Passing Arrays

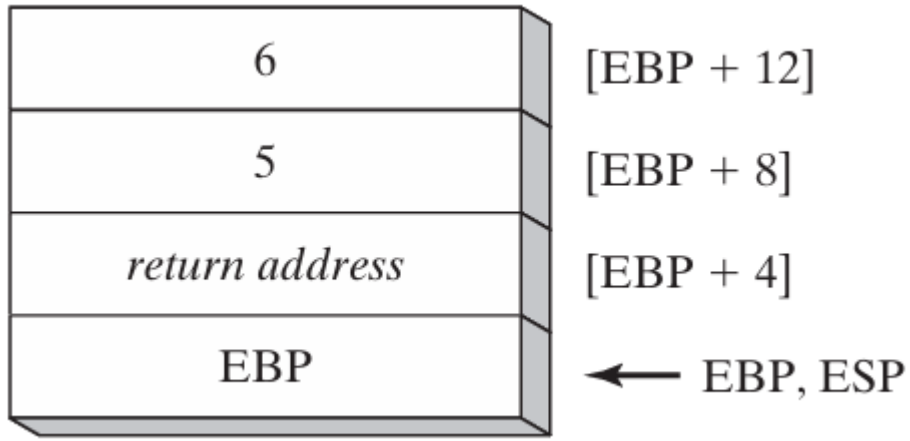
High-level languages always pass arrays to subroutines by reference.
That is, they push the address of an array on the stack.

```
.data
array  DWORD 50 DUP(?)
.code
push   OFFSET array
call   ArrayFill
```

Accessing Stack Parameters

AddTwo Example

```
int AddTwo( int x, int y )
{
     return x + y;
}
```




```
AddTwo PROC
     push  ebp
     mov   ebp,esp                ; base of stack frame
     mov   eax,[ebp + 12]         ; second parameter
     add   eax,[ebp + 8]          ; first parameter
     pop   ebp
     ret
AddTwo ENDP
```

Explicit Stack Parameters

When stack parameters are referenced with expressions such as [ebp + 8], we call them explicit stack parameters.
The reason for this term is that the assembly code explicitly states the offset of the parameter as a constant value. Some programmers define symbolic constants to represent the explicit stack parameters, to make their code easier to read

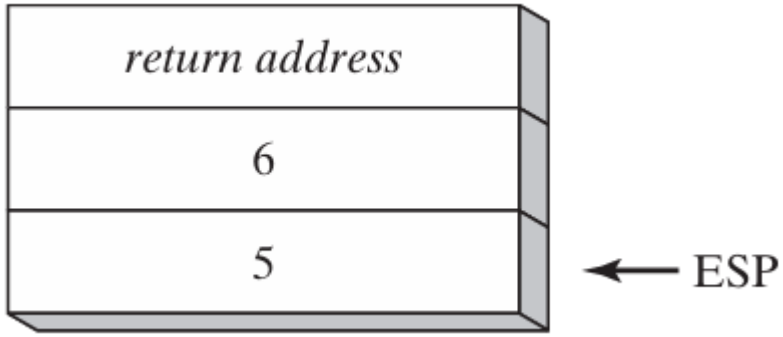
```
y_param EQU [ebp + 12]
x_param EQU [ebp + 8]

AddTwo PROC
    push  ebp
    mov   ebp,esp
    mov   eax,y_param
    add   eax,x_param
    pop   ebp
    ret
AddTwo ENDP
```

Cleaning Up the Stack

```
main PROC
     call  Example1
     exit
main ENDP

Example1 PROC
     push  6
     push  5
     call  AddTwo
     ret
Example1 ENDP
```




```
; stack is corrupted!
```

```
Example1 PROC
        push 6
        push 5
        call AddTwo
        add esp, 8      ; 2개 * 4바이트 스택 정리
        ret
Example1 ENDP
```

32-Bit Calling Conventions

The C Calling Convention

The C calling convention is used by the C and C++ programming languages. Subroutine parameters are pushed on the stack in reverse order, so a C program making a function call such as this will first push B on the stack, and then push A

```
AddTwo( A, B )


Example1 PROC
     push  6
     push  5
     call  AddTwo
     add   esp,8                    ; remove arguments from the stack
     ret
Example1 ENDP
```

STDCALL Calling Convention

Another common way to remove parameters from the stack is to use a convention named STDCALL. In the following AddTwo procedure, we supply an integer parameter to the RET instruction, which in turn adds 8 to ESP after returning to the calling procedure. The integer must equal the number of bytes of stack space consumed by the procedure's parameters

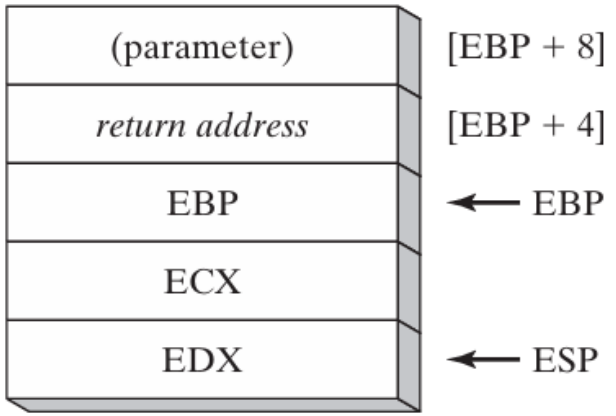
```
AddTwo PROC
     push  ebp
     mov   ebp,esp                ; base of stack frame
     mov   eax,[ebp + 12]         ; second parameter
     add   eax,[ebp + 8]          ; first parameter
     pop   ebp
     ret   8                      ; clean up the stack
AddTwo ENDP
```

Saving and Restoring Registers

Subroutines often save the current contents of registers on the stack before modifying them.

```
MySub PROC
     push  ebp                    ; save base pointer
     mov   ebp,esp                ; base of stack frame
     push  ecx
     push  edx                    ; save EDX
     mov   eax,[ebp+8]            ; get the stack parameter
     .
     .
     pop   edx                    ; restore saved registers
     pop   ecx
     pop   ebp                    ; restore base pointer
     ret                          ; clean up the stack
MySub ENDP
```

return address(EBP+4 위치에 있던 것)를
pop하여 호출한 함수로 복귀



Local Variables

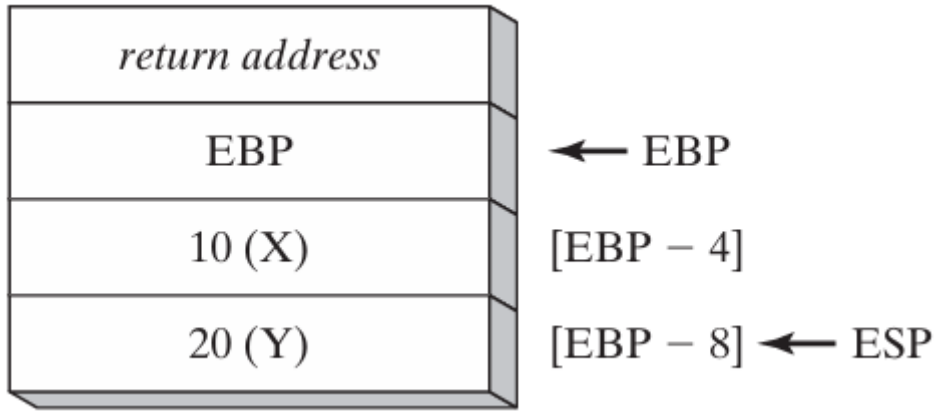
In high-level languages, variables created, used, and destroyed within a single subroutine are called local variables. Local variables are created on the runtime stack, usually below the base pointer (EBP).

Example

```
void MySub()
{
    int X = 10;
    int Y = 20;
}
```

Variable	Bytes	Stack Offset
X	4	EBP − 4
Y	4	EBP − 8

```
MySub PROC
     push  ebp
     mov   ebp,esp
     sub   esp,8                          ; create locals
     mov   DWORD PTR [ebp-4],10           ; X
     mov   DWORD PTR [ebp-8],20           ; Y
     mov   esp,ebp                        ; remove locals from stack
     pop   ebp
     ret
MySub ENDP
```



Before finishing, the function resets the stack pointer by assigning it the value of EBP. The effect is to release the local variables from the stack

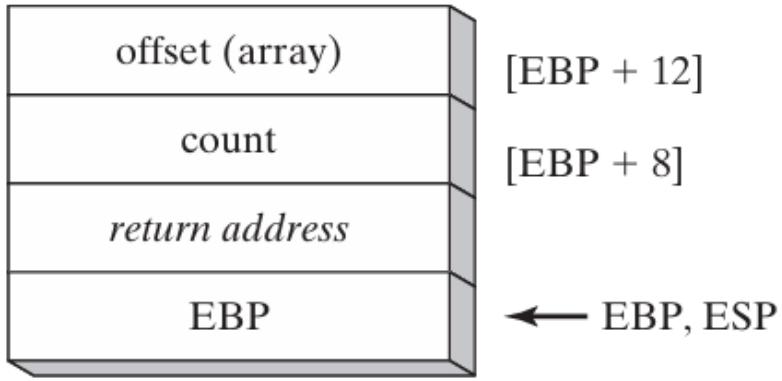
```
     mov   esp,ebp                        ; remove locals from stack
```

Reference Parameters

Reference parameters are usually accessed by procedures using base-offset addressing (from EBP).

ArrayFill Example

fills an array with a pseudorandom sequence of 16-bit integers.




```
ArrayFill PROC
     push  ebp
     mov   ebp,esp
     pushad                        ; save registers
     mov   esi,[ebp+12]            ; offset of array
     mov   ecx,[ebp+8]             ; array length
     cmp   ecx,0                   ; ECX == 0?
     je    L2                      ; yes: skip over loop
L1:
     mov   eax,10000h              ; get random 0 - FFFFh
     call  RandomRange             ; from the link library
     mov   [esi],ax                ; insert value in array
     add   esi,TYPE WORD           ; move to next element
     loop  L1
L2:  popad                         ; restore registers
     pop   ebp
     ret   8                       ; clean up the stack
ArrayFill ENDP
```

LEA Instruction

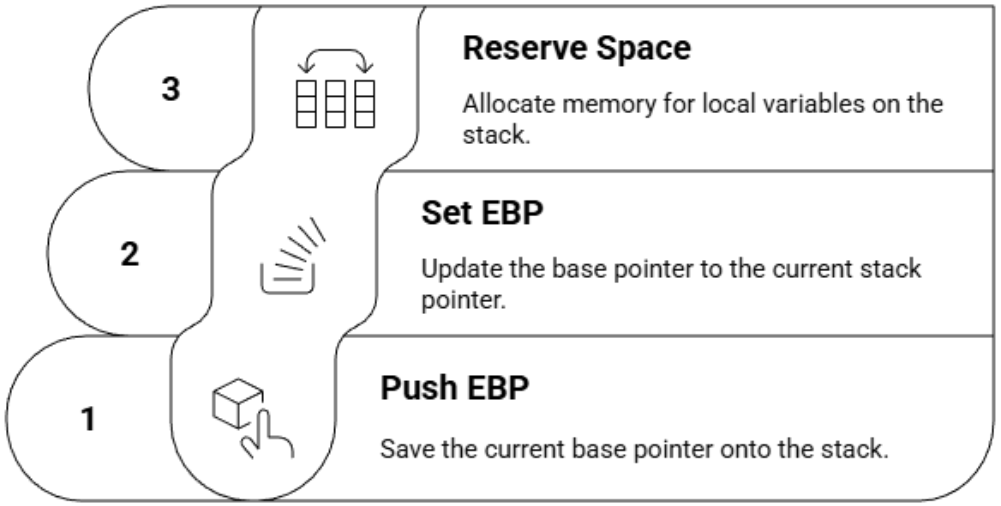
The LEA instruction returns the address of an indirect operand.

```
void makeArray( )
{
     char myString[30];
     for( int i = 0; i < 30; i++ )
          myString[i] = '*';
}
```

```
makeArray PROC
     push  ebp
     mov   ebp,esp
     sub   esp,32                ; myString is at EBP-30
     lea   esi,[ebp-30]          ; load address of myString
     mov   ecx,30                ; loop counter
L1:  mov   BYTE PTR [esi],'*'    ; fill one position
     inc   esi                   ; move to next
     loop  L1                    ; continue until ECX = 0
     add   esp,32                ; remove the array (restore ESP)
     pop   ebp
     ret
makeArray ENDP
```

ENTER and LEAVE Instructions

The ENTER instruction automatically creates a stack frame for a called procedure.
It reserves stack space for local variables and saves EBP on the stack.



ENTER has two operands: The first is a constant specifying the number of bytes of stack space to reserve for local variables and the second specifies the lexical nesting level of the procedure.

```
ENTER  numbytes, nestinglevel
```

Example 1

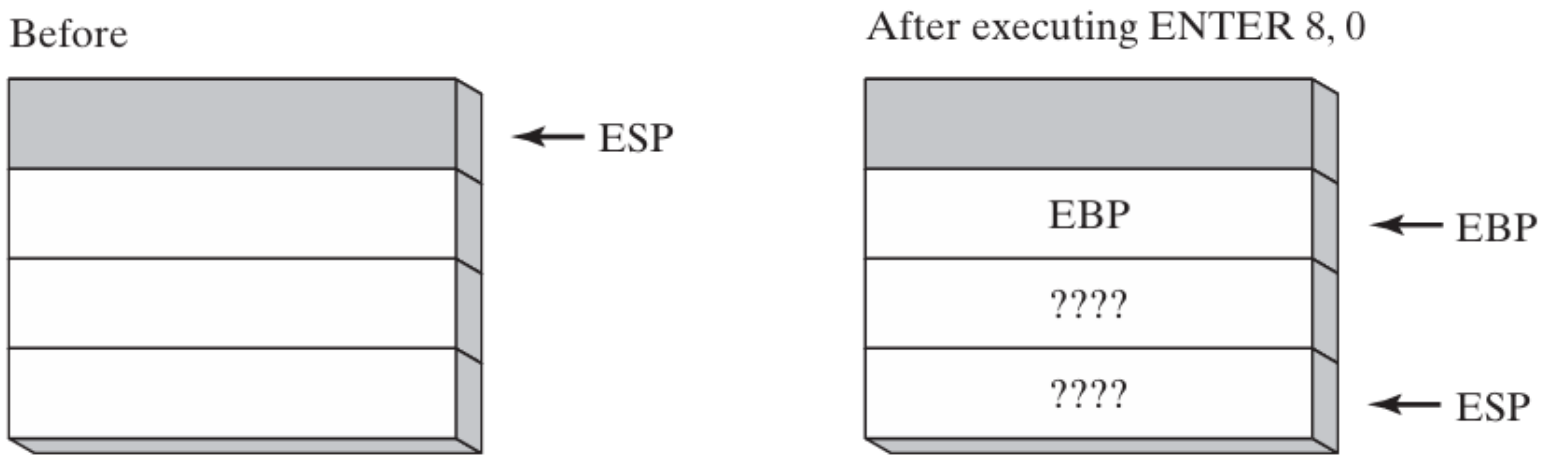
```
MySub PROC
     enter 0,0
```

```
MySub PROC
     push  ebp
     mov   ebp,esp
```

Example 2

```
MySub PROC
     enter 8,0
```

```
MySub PROC
     push  ebp
     mov   ebp,esp
     sub   esp,8
```



If you use the ENTER instruction, it is strongly advised that you also use the LEAVE instruction at the end of the same procedure.

LEAVE Instruction

The LEAVE instruction terminates the stack frame for a procedure. It reverses the action of a previous ENTER instruction by restoring ESP and EBP to the values they were assigned when the procedure was called.

```
MySub PROC
     enter 8,0
     .
     .
     leave
     ret
MySub ENDP
```

```
MySub PROC
     push  ebp
     mov   ebp,esp
     sub   esp,8
     .
     .
     mov   esp,ebp
     pop   ebp
     ret
MySub ENDP
```

LOCAL Directive

LOCAL declares one or more local variables by name, assigning them size attributes. (ENTER, on the other hand, only reserves a single unnamed block of stack space for local variables.)

```
LOCAL varlist
      label:type
```

Examples

```
MySub PROC
    LOCAL var1:BYTE

BubbleSort PROC
    LOCAL temp:DWORD, SwapFlag:BYTE

Merge PROC
    LOCAL pArray:PTR WORD

    LOCAL TempArray[10]:DWORD
```

Stack Frames

Recursion

A recursive subroutine is one that calls itself, either directly or indirectly.

Endless Recursion

```
; Endless Recursion                    (Endless.asm)

INCLUDE Irvine32.inc
.data
endlessStr BYTE "This recursion never stops",0
.code
main PROC
     call  Endless
     exit
main ENDP

Endless PROC
     mov   edx,OFFSET endlessStr
     call  WriteString
     call  Endless
     ret                          ; never executes
Endless ENDP
END main
```

Recursively Calculating a Sum

Useful recursive subroutines always contain a terminating condition.

```
; Sum of Integers              (RecursiveSum.asm)

INCLUDE Irvine32.inc
.code
main PROC
     mov   ecx,5               ; count = 5
     mov   eax,0               ; holds the sum
     call  CalcSum             ; calculate sum
L1:  call  WriteDec            ; display EAX
     call  Crlf                ; new line
     exit
main ENDP
```

```
;-----------------------------------------------------
CalcSum PROC
; Calculates the sum of a list of integers
; Receives: ECX = count
; Returns: EAX = sum
;-----------------------------------------------------
     cmp   ecx,0               ; check counter value
     jz    L2                  ; quit if zero
     add   eax,ecx             ; otherwise, add to sum
     dec   ecx                 ; decrement counter
     call  CalcSum             ; recursive call
     L2:  ret
     CalcSum ENDP
     end Main
```

The first two lines of CalcSum check the counter and exit the procedure when ECX 0. The code bypasses further recursive calls. When the RET instruction is reached for the first time, it returns to the previous call to CalcSum, which returns to its previous call, and so on.

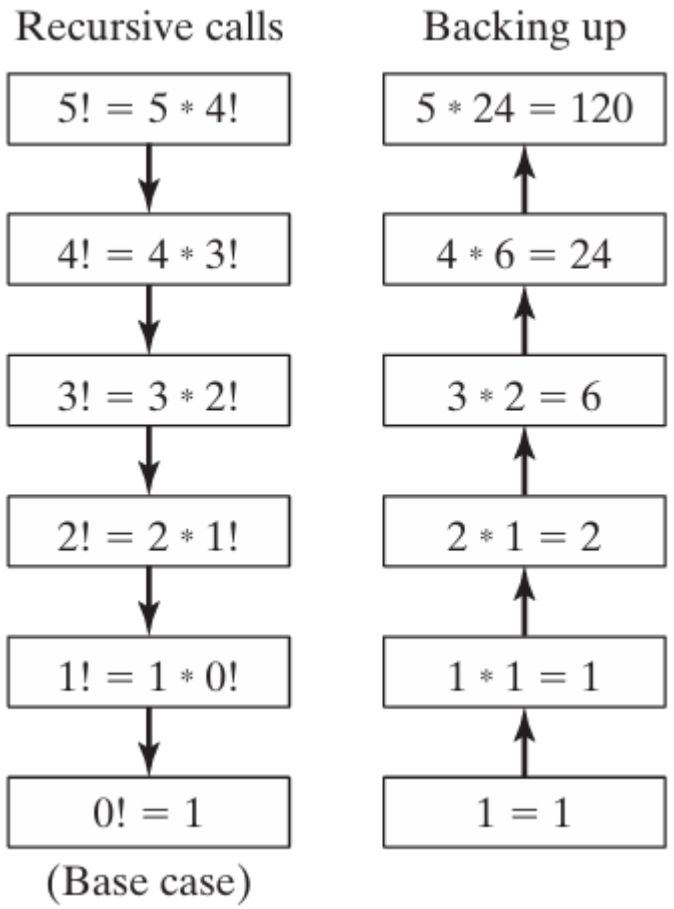
Table 8-1 Stack Frame and Registers (CalcSum).

Pushed on Stack	Value in ECX	Value in EAX
L1	5	0
L2	4	5
L2	3	9
L2	2	12
L2	1	14
L2	0	15

Calculating a Factorial

Recursive subroutines often store temporary data in stack parameters. When the recursive calls unwind, the data saved on the stack can be useful.

```
int function factorial(int n)
{
      if(n == 0)
        return 1;
      else
        return n * factorial(n−1);
}
```

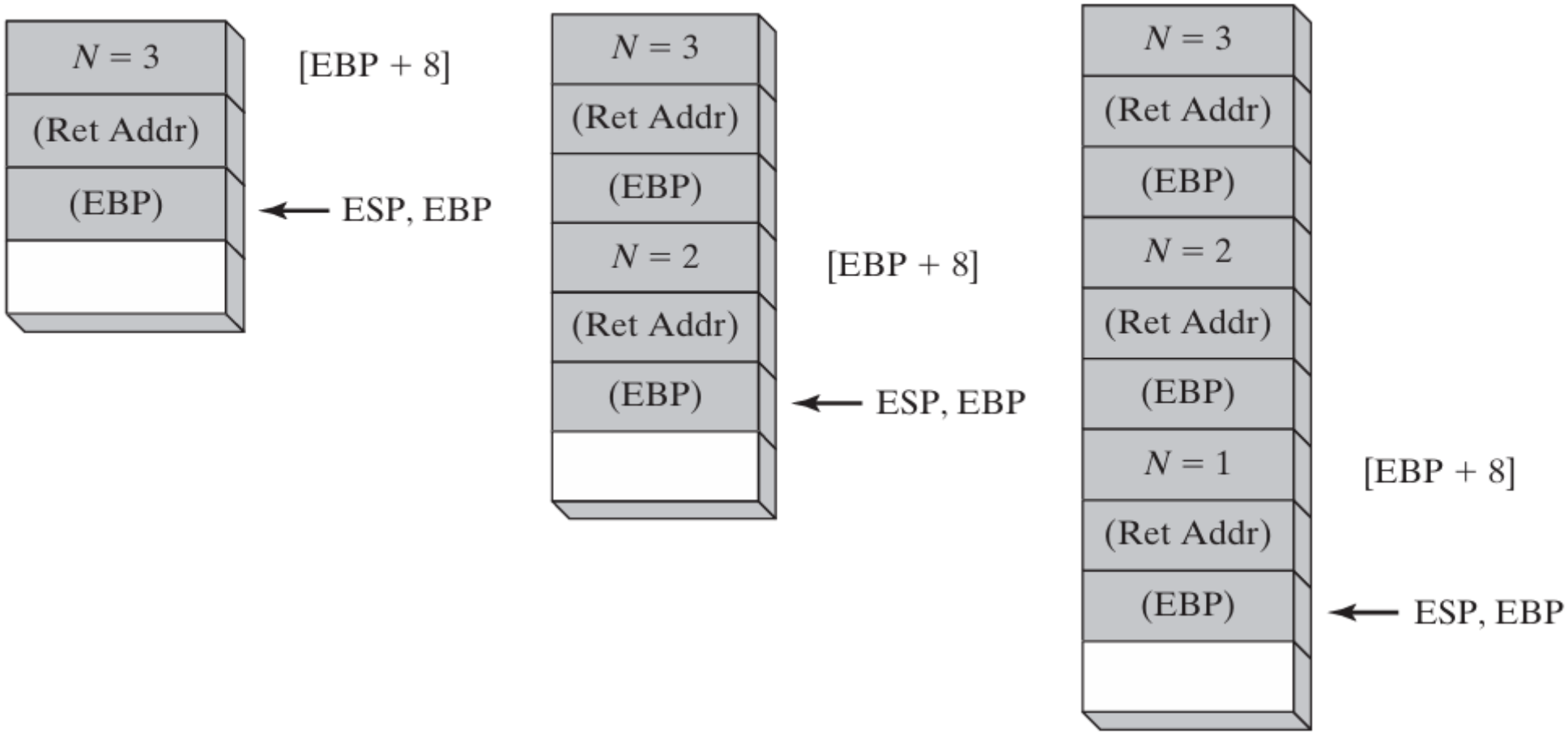


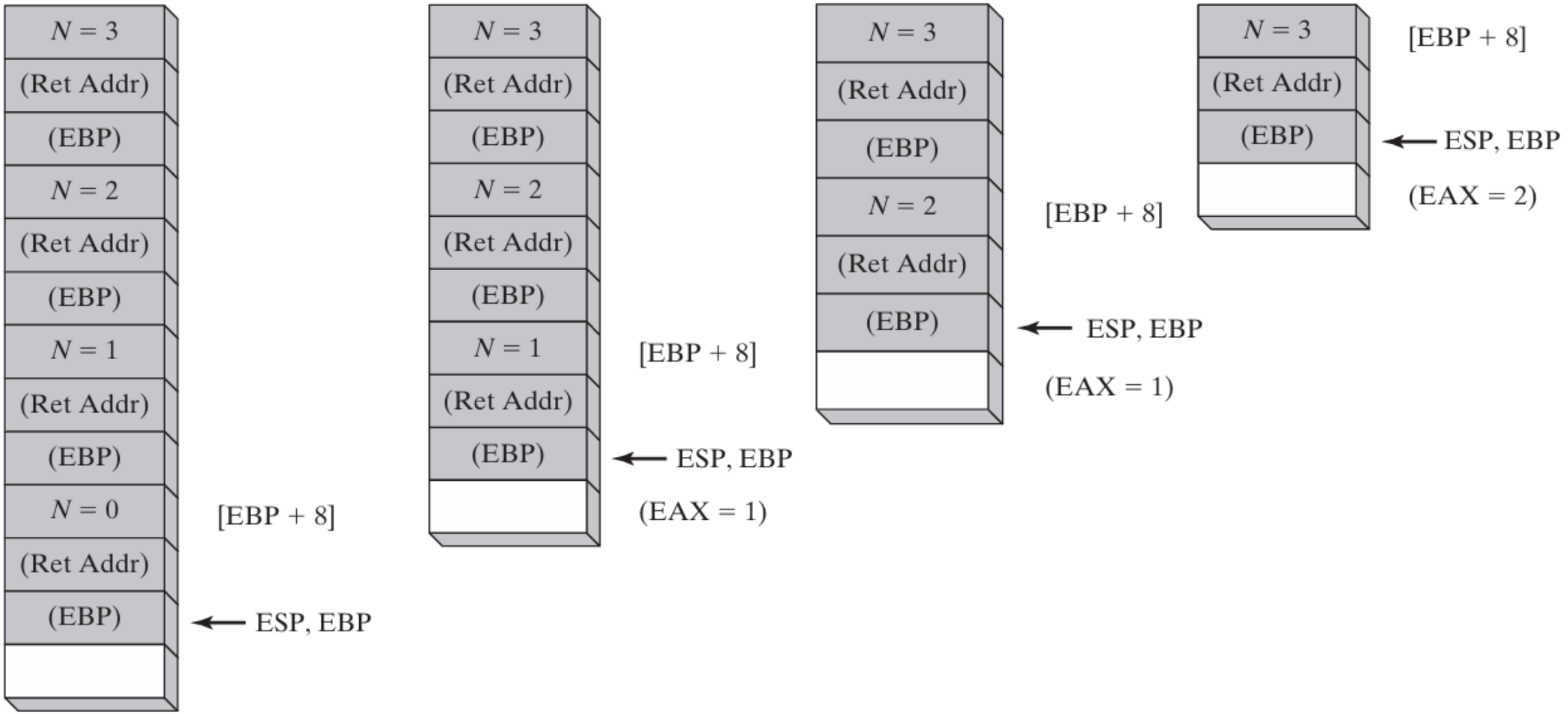
Example Program

The following assembly language program contains a procedure named Factorial that uses recursion to calculate a factorial. We pass n (an unsigned integer between 0 and 12) on the stack to the Factorial procedure, and a value is returned in EAX.

```
; Calculating a Factorial (Fact.asm)
INCLUDE Irvine32.inc
.code
main PROC
        push 5                  ; calc 5!
        call Factorial          ; calculate factorial (EAX)
        call WriteDec           ; display it
        call Crlf
        exit
main ENDP
;-----------------------------------------------
Factorial PROC
; Calculates a factorial.
; Receives: [ebp+8] = n, the number to calculate
; Returns: eax = the factorial of n
;-----------------------------------------------
        push ebp
        mov ebp,esp
        mov eax,[ebp+8]         ; get n
        cmp eax,0               ; n > 0?
        ja L1                   ; yes: continue
        mov eax,1               ; no: return 1 as the value of 0!
        jmp L2                  ; and return to the caller
L1:     dec eax
        push eax                ; Factorial(n-1)
        call Factorial
; Instructions from this point on execute when each
; recursive call returns.
ReturnFact:
        mov ebx,[ebp+8]         ; get n
        mul ebx                 ; EDX:EAX = EAX * EBX
L2:     pop ebp                 ; return EAX
        ret 4                   ; clean up stack
Factorial ENDP
END main
```

Let's examine the Factorial procedure more closely by tracking a call to it with an initial value of N = 3.




N = 3
(Ret Addr)
(EBP)
N = 2
(Ret Addr)
(EBP)
N = 1
(Ret Addr)
(EBP)
N = 0
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
N = 3
(Ret Addr)
(EBP)
N = 2
(Ret Addr)
(EBP)
N = 1
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
(EAX = 1)
N = 3
(Ret Addr)
(EBP)
N = 2
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
(EAX = 1)
N = 3
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
(EAX = 2)

Recursion

INVOKE Directive

The INVOKE directive, only available in 32-bit mode, pushes arguments on the stack (in the order specified by the MODEL directive's language specifier) and calls a procedure. INVOKE is a convenient replacement for the CALL instruction because it lets you pass multiple arguments using a single line of code.

```
INVOKE procedureName [, argumentList]
```

```
push  TYPE array
push  LENGTHOF array
push  OFFSET array
call  DumpArray
```

```
INVOKE DumpArray, OFFSET array, LENGTHOF array, TYPE array
```

INVOKE permits almost any number of arguments, and individual arguments can appear on separate source code lines.

```
INVOKE DumpArray,                ; displays an array
     OFFSET array,               ; points to the array
     LENGTHOF array,             ; the array length
     TYPE array                  ; array component size
```

Table 8-2 Argument types used with INVOKE.

Type	Examples
Immediate value	10, 3000h, OFFSET mylist, TYPE array
Integer expression	(10 * 20), COUNT
Variable	myList, array, myWord, myDword
Address expression	[myList+2], [ebx + esi]
Register	eax, bl, edi
ADDR *name*	ADDR myList
OFFSET *name*	OFFSET myList

ADDR Operator

The ADDR operator, also available in 32-bit mode, can be used to pass a pointer argument when calling a procedure using INVOKE. The following INVOKE statement, for example, passes the address of myArray to the FillArray procedure

```
INVOKE FillArray, ADDR myArray
INVOKE mySub, ADDR [ebp+12]     ; error
```

The argument passed to ADDR must be an assembly time constant.

```
mov  esi, ADDR myArray          ; error
```

The ADDR operator can only be used in conjunction with INVOKE.

Example

```
.data
Array DWORD 20 DUP(?)
.code
...
INVOKE Swap,
    ADDR  Array,
    ADDR  [Array+4]
```

```
push  OFFSET Array+4
push  OFFSET Array
call  Swap
```

PROC Directive

The PROC directive is a directive used to define a procedure.

```
label PROC [attributes] [USES reglist], parameter_list
```



```
[distance] [langtype] [visibility] [prologuearg]
```

Table 8-3 Attributes field in the PROC directive.

Attribute	Description
distance	NEAR or FAR. Indicates the type of RET instruction (RET or RETF) generated by the assembler.
langtype	Specifies the calling convention (parameter passing convention) such as C, PASCAL, or STDCALL. Overrides the language specified in the .MODEL directive.
visibility	Indicates the procedure's visibility to other modules. Choices are PRIVATE, PUBLIC (default), and EXPORT. If the visibility is EXPORT, the linker places the procedure's name in the export table for segmented executables. EXPORT also enables PUBLIC visibility.
prologuearg	Specifies arguments affecting generation of prologue and epilogue code.

Parameter Lists

The PROC directive permits you to declare a procedure with a comma-separated list of named parameters.

```
label PROC [attributes] [USES reglist],
    parameter_1,
    parameter_2,
    .
    .
    parameter_n
```

```
label PROC [attributes], parameter_1, parameter_2, ..., parameter_n
```

```
paramName:type
```

Example 1

The AddTwo procedure receives two doubleword values and returns their sum in EAX

```
AddTwo PROC,
     val1:DWORD,
     val2:DWORD
     mov   eax,val1
     add   eax,val2
     ret
AddTwo ENDP
```

```
AddTwo PROC
     push  ebp
     mov   ebp, esp
     mov   eax,dword ptr [ebp+8]
     add   eax,dword ptr [ebp+0Ch]
     leave
     ret   8
AddTwo ENDP
```

Example 2

The FillArray procedure receives a pointer to an array of bytes

```
FillArray PROC,
     pArray:PTR BYTE
     . . .
FillArray ENDP
```

Example 3

The Swap procedure receives two pointers to doublewords

```
Swap PROC,
     pValX:PTR DWORD,
     pValY:PTR DWORD
     . . .
Swap ENDP
```

INVOKE, ADDR, PROC and PROTO

Example 4

The Read_File procedure receives a byte pointer named pBuffer. It has a local doubleword variable named fileHandle, and it saves two registers on the stack (EAX and EBX)

```
Read_File PROC USES eax ebx,
     pBuffer:PTR BYTE
     LOCAL fileHandle:DWORD

     mov   esi,pBuffer
     mov   fileHandle,eax
     .
     .
     ret
Read_File ENDP
```

```
Read_File PROC
     push  ebp
     mov   ebp,esp
     add   esp,0FFFFFFFCh              ; create fileHandle
     push  eax                         ; save EAX
     push  ebx                         ; save EBX
     mov   esi,dword ptr [ebp+8]       ; pBuffer   parameter
     mov   dword ptr [ebp-4],eax       ; fileHandle
     pop   ebx
     pop   eax
     leave
     ret   4
Read_File ENDP
```

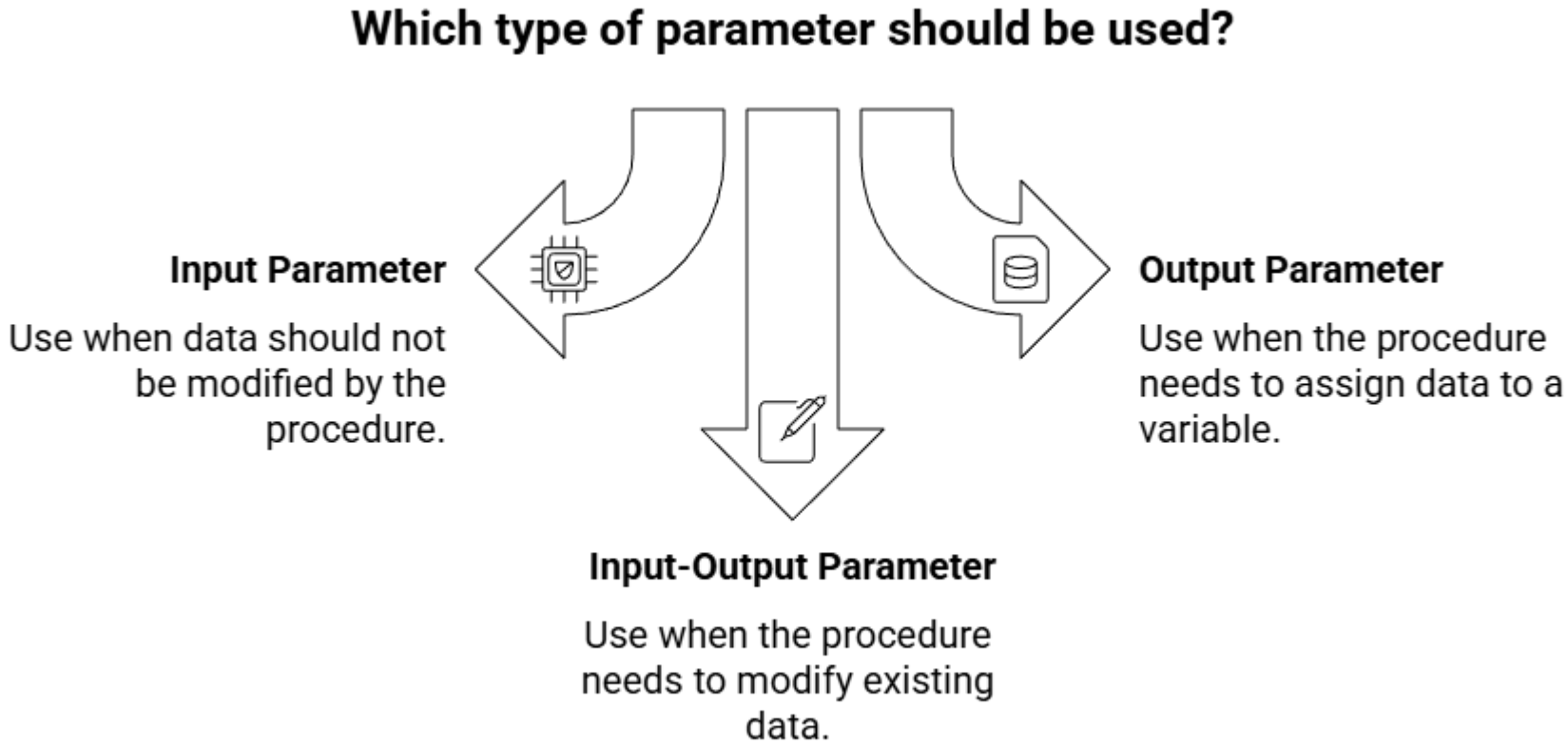
RET Instruction Modified by PROC

When PROC is used with one or more parameters and STDCALL is the default protocol, MASM generates the following entry and exit code, assuming PROC has n parameters

```
push  ebp
mov   ebp,esp
.
.
leave
ret   (n*4)
```

Parameter Classifications

Procedure parameters are usually classified according to the direction of data transfer between the calling program and the called procedure



Example: Exchanging Two Integers

```
; Swap Procedure Example                    (Swap.asm)
INCLUDE Irvine32.inc
Swap PROTO, pValX:PTR DWORD, pValY:PTR DWORD
.data
Array DWORD 10000h,20000h
.code
main PROC
        ; Display the array before the exchange:
        mov esi,OFFSET Array
        mov ecx,2                           ; count = 2
        mov ebx,TYPE Array
        call DumpMem                        ; dump the array values
        INVOKE Swap, ADDR Array, ADDR [Array+4]
        ; Display the array after the exchange:
        call DumpMem
        exit
main ENDP
```

INVOKE는 다음을 자동으로 수행해 주는 함수 호출용 매크로
1. 파라미터(push) 순서를 자동으로 맞춰서 스택에 넣고
2. call 명령으로 함수 실행
3. 호출 규칙(stdcall, cdecl 등)에 따라 스택 정리까지 자동 처리

INVOKE, ADDR, PROC and PROTO

```
;-----------------------------------------------------
Swap PROC USES eax esi edi,
        pValX:PTR DWORD,        ; pointer to first integer
        pValY:PTR DWORD         ; pointer to second integer
;
; Exchange the values of two 32-bit integers
; Returns: nothing
;-----------------------------------------------------
        mov esi,pValX           ; get pointers
        mov edi,pValY
        mov eax,[esi]           ; get first integer
        xchg eax,[edi]          ; exchange with second
        mov [esi],eax           ; replace first integer
        ret                     ; PROC generates RET 8 here
Swap ENDP
END main
```

Debugging Tips

Argument Size Mismatch

```
.data
DoubleArray DWORD 10000h,20000h
.code
INVOKE Swap, ADDR [DoubleArray + 0], ADDR [DoubleArray + 1]
```

Passing the Wrong Type of Pointer

```
.data
ByteArray BYTE 10h,20h,30h,40h,50h,60h,70h,80h
.code
INVOKE Swap, ADDR [ByteArray + 0], ADDR [ByteArray + 1]
```

The Swap procedure expects to receive two doubleword pointers.
Suppose we inadvertently pass it pointers to bytes.

Passing Immediate Values

```
Sub2 PROC, dataPtr:PTR WORD
     mov   esi,dataPtr          ; get the address
     mov   WORD PTR [esi],0     ; dereference, assign zero
     ret
Sub2 ENDP


INVOKE  Sub2, 1000h
```

The Sub2 procedure receives 1000h as a pointer value and dereferences memory location 1000h

runtime error

WriteStackFrame Procedure

```
main PROC
      mov eax, 0EAEAEAEAh
      mov ebx, 0EBEBEBEBh
      INVOKE myProc, 1111h, 2222h ; pass two integer arguments
      exit
main ENDP

myProc PROC USES eax ebx,
     x: DWORD, y: DWORD
     LOCAL a:DWORD, b:DWORD

     PARAMS = 2
     LOCALS = 2
     SAVED_REGS = 2
     mov a,0AAAAh
     mov b,0BBBBh
     INVOKE WriteStackFrame, PARAMS, LOCALS, SAVED_REGS
```

The Irvine32 library contains a useful procedure named WriteStackFrame that displays the contents of the current procedure's stack frame. It shows the procedure's stack parameters, return address, local variables, and saved registers.

INVOKE, ADDR, PROC and PROTO

```
Stack Frame

00002222 ebp+12 (parameter)
00001111 ebp+8 (parameter)
00401083 ebp+4 (return address)
0012FFF0 ebp+0 (saved ebp) <--- ebp
0000AAAA ebp-4 (local variable)
0000BBBB ebp-8 (local variable)
EAEAEAEA ebp-12 (saved register)
EBEBEBEB ebp-16 (saved register) <--- esp
```

Creating Multimodule Programs

Each module is assembled independently, so a change to one module's source code only requires reassembling the single module. The linker combines all assembled modules (OBJ files) into a sin gle executable file rather quickly. Linking large numbers of object modules requires far less time than assembling the same number of source code files.



Hiding and Exporting Procedure Names

By default, MASM makes all procedures public, permitting them to be called from any other module in the same program. You can override this behavior using the PRIVATE qualifier

```
mySub PROC PRIVATE
```

By making procedures private, you use the principle of encapsulation to hide procedures inside modules and avoid potential name clashes when procedures in different modules have the same names.

OPTION PROC:PRIVATE Directive

Another way to hide procedures inside a source module is to place the OPTION PROC:PRIVATE directive at the top of the file. All procedures become private by default. Then, you use the PUBLIC directive to identify any procedures you want to export:

```
OPTION PROC:PRIVATE
PUBLIC mySub
```

The PUBLIC directive takes a comma-delimited list of names:

```
PUBLIC sub1, sub2, sub3
```

Alternatively, you can designate individual procedures as public:

```
mySub PROC PUBLIC
.
mySub ENDP
```

If you use OPTION PROC:PRIVATE in your program's startup module, be sure to designate your startup procedure (usually main) as PUBLIC, or the operating system's loader will not be able to find it.

```
main PROC PUBLIC
```

Calling External Procedures

The EXTERN directive, used when calling a procedure outside the current module, identifies the procedure's name and stack frame size.

```
INCLUDE Irvine32.inc
EXTERN sub1@0:PROC
.code
main PROC
     call  sub1@0
     exit
main ENDP
END main
```

The @n suffix at the end of a procedure name identifies the total stack space used by declared parameters

INVOKE, ADDR, PROC and PROTO

```
AddTwo PROC,
     val1:DWORD,
     val2:DWORD
     . . .
AddTwo ENDP
```

EXTERN AddTwo@8:PROC

If you declare a procedure using the extended PROC directive, add 4 bytes for every parameter.

```
AddTwo PROTO,
     val1:DWORD,
     val2:DWORD
```

Using Variables and Symbols across Module Boundaries

Exporting Variables and Symbols

Variables and symbols are, by default, private to their enclosing modules.
You can use the PUBLIC directive to export specific names

```
PUBLIC count, SYM1
SYM1 = 10
.data
count DWORD 0
```

Accessing External Variables and Symbols

```
EXTERN name : type
```

```
EXTERN one:WORD, two:SDWORD, three:PTR BYTE, four:ABS
```

Using an INCLUDE File with EXTERNDEF

MASM has a useful directive named EXTERNDEF that takes the place of both PUBLIC and EXTERN.

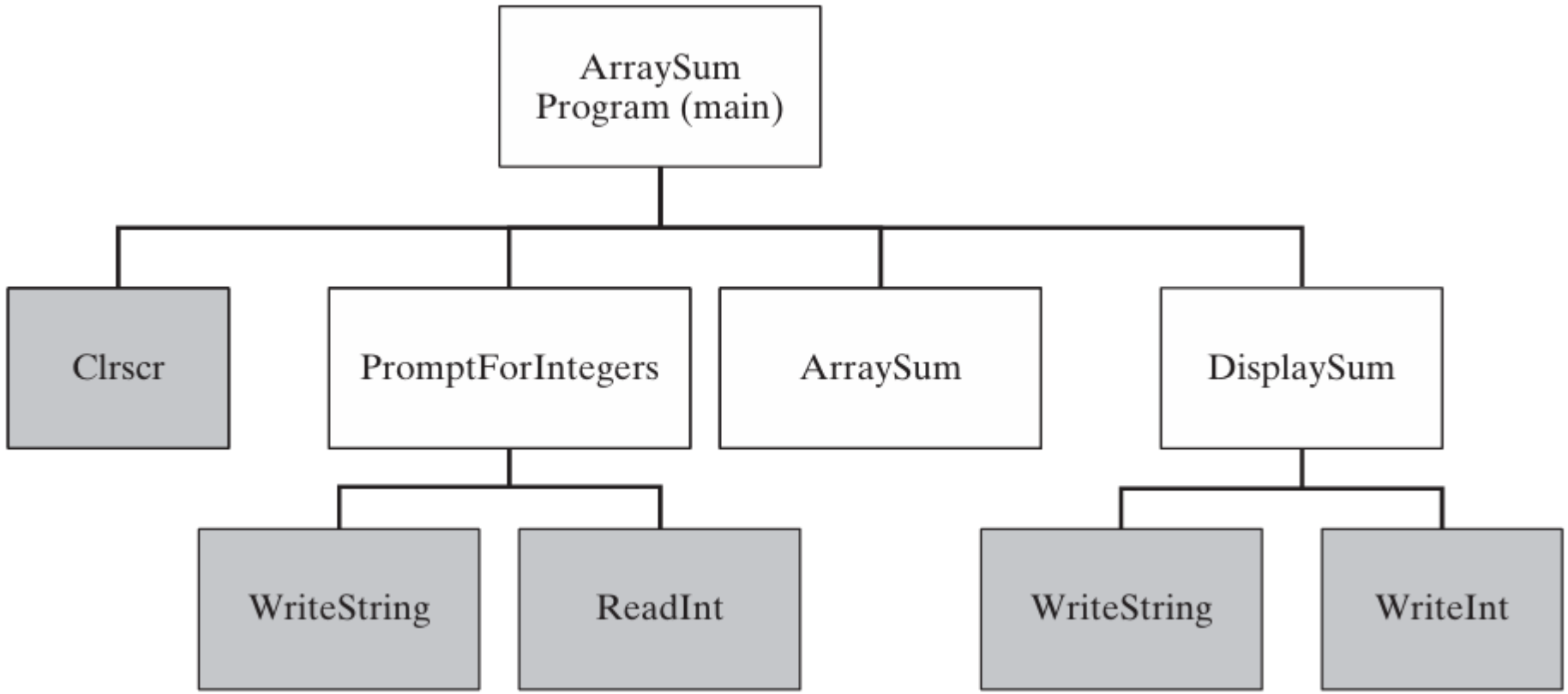
```
; vars.inc
EXTERNDEF count:DWORD, SYM1:ABS
```

```
; sub1.asm
.386
.model flat,STDCALL
INCLUDE vars.inc
SYM1 = 10
.data
count DWORD 0
END
```

```
; main.asm
.386
.model flat,stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword
INCLUDE vars.inc
.code
main PROC
     mov   count,2000h
     mov   eax,SYM1
     INVOKE ExitProcess,0
main ENDP
END main
```

Example: ArraySum Program

Figure 8–5 Structure chart, ArraySum program.



INVOKE, ADDR, PROC and PROTO

Advanced Procedures

8.1 Introduction

8.2 Stack Frames
- 8.2.1 Stack Parameters
- 8.2.2 Disadvantages of Register Parameters
- 8.2.3 Accessing Stack Parameters
- 8.2.4 32-Bit Calling Conventions
- 8.2.5 Local Variables
- 8.2.6 Reference Parameters
- 8.2.7 LEA Instruction
- 8.2.8 ENTER and LEAVE Instructions
- 8.2.9 LOCAL Directive
- 8.2.10 The Microsoft x64 Calling Convention
- 8.2.11 Section Review

8.3 Recursion
- 8.3.1 Recursively Calculating a Sum
- 8.3.2 Calculating a Factorial
- 8.3.3 Section Review

8.4 INVOKE, ADDR, PROC, and PROTO
- 8.4.1 INVOKE Directive
- 8.4.2 ADDR Operator
- 8.4.3 PROC Directive
- 8.4.4 PROTO Directive
- 8.4.5 Parameter Classifications
- 8.4.6 Example: Exchanging Two Integers
- 8.4.7 Debugging Tips
- 8.4.8 WriteStackFrame Procedure
- 8.4.9 Section Review

8.5 Creating Multimodule Programs
- 8.5.1 Hiding and Exporting Procedure Names
- 8.5.2 Calling External Procedures
- 8.5.3 Using Variables and Symbols across Module Boundaries
- 8.5.4 Example: ArraySum Program
- 8.5.5 Creating the Modules Using Extern
- 8.5.6 Creating the Modules Using INVOKE and PROTO
- 8.5.7 Section Review

8.6 Advanced Use of Parameters (Optional Topic)
- 8.6.1 Stack Affected by the USES Operator
- 8.6.2 Passing 8-Bit and 16-Bit Arguments on the Stack
- 8.6.3 Passing 64-Bit Arguments
- 8.6.4 Non-Doubleword Local Variables

8.7 Java Bytecodes (Optional Topic)
- 8.7.1 Java Virtual Machine
- 8.7.2 Instruction Set
- 8.7.3 Java Disassembly Examples
- 8.7.4 Example: Conditional Branch

8.8 Chapter Summary

8.9 Key Terms
- 8.9.1 Terms
- 8.9.2 Instructions, Operators, and Directives

8.10 Review Questions and Exercises
- 8.10.1 Short Answer
- 8.10.2 Algorithm Workbench

8.11 Programming Exercises

reference

https://www.asmirvine.com/
https://intelligentcm.tistory.com/category/Assembly

Review questions and Exercises

&

Programming exercises

Github upload